Elone Prism: Merkleized Channels as Proof-First State Machines

Deterministic Settlement and Partition Conservation for Native Off-Chain State

Arthur Zhang*
Tondi Foundation

March, 2026

Abstract

Existing off-chain channel systems often conflate three logically distinct concerns: the representation of channel state, the source of transaction fees, and the authorization boundary for unilateral settlement. The current Elone design already commits channel state through `balances_root` and `ptlcs_root`, but still relies on plaintext witnesses, local plaintext authority, and a `max_fee` field embedded in the channel signing domain. This paper proposes a coherent refinement that completes the Merkleization of state and simultaneously eliminates channel-paid fees.

Problem Statement: Partial Merkleization is structurally unstable. If roots exist but validators still require plaintext state, the system inherits both the complexity of commitment-based state and the privacy leakage of plaintext-first execution. Likewise, removing `max_fee` without introducing a stronger conservation rule would weaken the fee boundary rather than simplify it.

Technical Contributions: We make seven contributions. First, we define a **dual-subtree state commitment model** with separate balance and PTLC subtrees plus an optional aggregate root, together with a canonical `state_header_commitment`. Second, we introduce a **sponsor-paid fee model** in which channel value never pays fees. Third, we formalize **partition conservation**: exact conservation of channel-owned value and fee extraction exclusively from sponsor-owned value. Fourth, we distinguish **deterministic unilateral settlement** from **explicitly authorized cooperative settlement**, thereby repairing the output-authorization boundary of settlement flows. Fifth, we propose a **proof-first witness and storage model** in which local authority is expressed through roots, proofs, signatures, and encrypted leaf bundles rather than long-lived plaintext state. Sixth, we define **transition-footprint semantics** and show why proof-first validation is locality-preserving rather than full-state-reconstructive. Seventh, we characterize the **state package** as the correct recovery object: richer than a bare root, leaner than a permanent plaintext dump.

Formal Consequences: We show that partition conservation implies zero channel-paid fees and that deterministic settlement descriptors yield a unique unilateral settlement output set, up to canonical serialization. Under an explicit simple-proof model, we also show that proof-first verification is naturally local in the touched state footprint and that a bare state root is insufficient for unilateral recovery. These results replace fee-cap reasoning with a stronger invariant over value partitions and replace plaintext-authoritative recovery with a sharper notion of recoverable protocol meaning.

Paper Type: This draft should be read as a March 2026 protocol-design paper derived from the new Elone Merkle design memorandum. It is not a claim that the full design is already implemented, nor does it yet claim systems-level empirical closure. Rather, it provides a formalized architectural specification and migration discipline for upgrading Elone from a plaintext-assisted channel protocol to a proof-first, merkleized channel architecture.

*Corresponding author: Arthur Zhang (arthur@tondi.org)

Keywords: Payment Channels, Merkleized Channels, Partition Conservation, Proof-First Witnesses, Deterministic Settlement, Elone Protocol

Contents

1 Introduction and Motivation **5**
1.1 Research Question . . . 5
1.2 Scope . . . 5
1.3 Why This Requires a Dedicated Elone Paper . . . 6

2 Problem Statement and Current Snapshot **6**
2.1 The Half-Merkle Problem . . . 6
2.2 The Fee-Boundary Problem . . . 6
2.3 The Settlement-Authorization Problem . . . 6
2.4 Current-vs-Proposed Interpretation . . . 7
2.5 Three Competing Truths in the Legacy Direction . . . 7

3 Research Contributions and Design Principles **7**
3.1 Main Contributions . . . 7
3.2 Design Principles . . . 8

4 System Model and Threat Assumptions **8**
4.1 Actors . . . 8
4.2 Adversarial Goals . . . 9
4.3 Protocol Invariants . . . 9

5 Formal Transition Semantics **9**
5.1 Channel Configuration . . . 9
5.2 Transition Footprint . . . 9
5.3 Locality of Verification . . . 10

6 Merkleized State Commitment Model **10**
6.1 Dual Subtrees and Optional Aggregate Root . . . 10
6.2 Tree-Internal vs. Tree-External Data . . . 11
6.3 State Carrier Reserve . . . 11
6.4 Canonical Leaf Discipline . . . 11
6.5 Proof Objects as First-Class Protocol Objects . . . 12

7 Sponsor-Paid Fees and Partition Conservation **12**
7.1 Initialization vs. Post-Funding Transitions . . . 12
7.2 Partitioned Value Semantics . . . 12
7.3 Partition Assignment Semantics . . . 13
7.4 Why Fee Caps Are Insufficient . . . 14
7.5 Why Sponsorship Remains Central Even If the Title Changes . . . 14

8 Transaction Semantics Under the New Model **14**
8.1 FUND . . . 14
8.2 Legacy vs. Elone Prism Semantics . . . 14
8.3 UPDATE . . . 15
8.4 SETTLE . . . 15
8.5 Settlement Descriptor as a Protocol Object . . . 16
8.6 A Minimal Descriptor Calculus . . . 16
8.7 Descriptor Commitment and Upgrade Boundaries . . . 16
8.8 Timed-Settle Derivation . . . 17
8.9 SPLICE . . . 17

9 Proof-First Witness and Local Persistence **17**
9.1 From Plaintext Authority to Commitment Authority . . . 17
9.2 Recommended Storage Layers . . . 18
9.3 State Package as the New Recovery Unit . . . 18
9.4 Privacy Interpretation . . . 18

10 Recoverability as a Protocol Property **19**
10.1 Bare Roots, Plaintext Dumps, and State Packages . . . 19
10.2 Authority Compression . . . 19
10.3 Failure Domains Under the New Model . . . 19

11 Signature Domains and Authorization Boundaries **20**
11.1 State-Header-Centric Signing . . . 20
11.2 UPDATE vNext . . . 20
11.3 SETTLE vNext . . . 20
11.4 SPLICE vNext . . . 20
11.5 Proof Commitments and Signature Minimality . . . 21

12 Evaluation Agenda and Measurable Claims **21**
12.1 What This Draft Does and Does Not Claim . . . 21
12.2 Hypotheses for Empirical Evaluation . . . 21
12.3 Minimum Benchmark Matrix . . . 21
12.4 Why This Matters . . . 22

13 Related Work and Positioning **22**
13.1 Payment Channels, Update-by-Replacement, and Factories . . . 22
13.2 Merkleized State and Witness-Carrying Validation . . . 22
13.3 Fee Sponsorship and Publication Economics . . . 22
13.4 What Elone Prism Adds Beyond Generic Merkleization . . . 22
13.5 Comparative Axes . . . 23
13.6 A Distinctive Research Claim . . . 23

14 Migration Roadmap and Open Engineering Questions **23**
14.1 Recommended Migration Order . . . 23
14.2 Open Question 1: Liveness Under Sponsor Dependence . . . 24
14.3 Open Question 2: Proof Size and DoS Envelope . . . 24
14.4 Open Question 3: Descriptor Upgrade Flexibility . . . 25
14.5 Open Question 4: Factory-Level Composition . . . 25

15 Conclusion **25**

1 Introduction and Motivation

Payment channel protocols require three properties simultaneously: low-latency off-chain state progression, enforceable unilateral settlement, and clear responsibility for transaction fees. In practice, these properties are frequently entangled. State is represented in one form for hashing and another for validation; fee budgets are embedded in signatures even when the intended policy is "the channel itself should never pay fees"; and settlement outputs are often only partially authorized.

The new Elone Merkle design note of March 12, 2026 exposes this entanglement with unusual clarity. The current Elone codebase is not a system with "no Merkle support." Rather, it is a system in which Merkle commitments already exist, but the protocol still behaves operationally as though plaintext channel state remains the authoritative witness. The result is a half-completed architecture: the system pays the conceptual cost of commitment-based state without fully obtaining its privacy, modularity, or proof-locality benefits.

This paper argues that the correct response is not a narrow local fix. Merkleization, sponsor-paid fees, and settlement authorization must be redesigned together. If only `max_fee` is removed, the fee boundary becomes weaker. If only state storage is Merkleized but consensus still consumes plaintext witnesses, the protocol becomes more complex without becoming cleaner. If settlement outputs remain weakly authorized, sponsor-funded fees simply move the ambiguity to another layer.

Our central thesis is therefore:

> *A proof-first channel architecture requires simultaneous separation of state commitments, value conservation, and fee sponsorship.*

This separation yields a more precise protocol language. Channel-owned value and sponsor-owned value become distinct partitions. Large, mutable collections move into Merkle subtrees. Hot metadata stays outside the tree. Unilateral settlement becomes descriptor-driven and deterministic. Cooperative settlement becomes explicitly committed. And the local authority model shifts from "latest plaintext snapshot" to "latest signed roots plus proofs and encrypted leaf material."

1.1 Research Question

The research question of this paper is not whether Merkle commitments can be applied to channel state in principle. That question has already been answered by the existence of the current dual-root Elone state format. The real question is sharper:

> *What protocol structure is required so that Merkleized state, sponsor-paid fees, and settlement authorization compose into a coherent channel system?*

1.2 Scope

This paper focuses on the Elone protocol family. It does not attempt a general treatment of all payment channel systems. Nor does it claim that the entire design is already implemented in production code. The intended contribution is a research-level protocol refinement and migration target for Elone itself.

Accordingly, the present draft should be judged primarily as a protocol-design paper rather than as a finished systems paper. Where we introduce formal objects and propositions, the goal is to make the design auditable and extensible; where we do not yet provide empirical measurements, we say so explicitly and turn those gaps into a benchmark agenda rather than smuggling them in as implied evidence.

1.3 Why This Requires a Dedicated Elone Paper

This Merkle refinement is not a minor appendix to the earlier Elone state-machine paper. It changes the meaning of three protocol objects:

1. **state**: from plaintext snapshots to commitment-authoritative state headers and proofs;
2. **fee responsibility**: from bounded channel expenditure to strict channel-value conservation;
3. **settlement authorization**: from builder-shaped output construction to descriptor-driven or root-bound authorization.

For that reason, this document is best read as a dedicated Elone paper in its own right. The earlier Elone paper explains why native channel semantics belong at the consensus layer. **Elone Prism** explains how the next-generation state, witness, fee, and settlement objects should be structured once Merkle commitments become first-class protocol artifacts.

2 Problem Statement and Current Snapshot

2.1 The Half-Merkle Problem

The current Elone state object already contains two cryptographic commitments:

- `balances_root`
- `ptlcs_root`

However, UPDATE and SETTLE validation still conceptually rely on plaintext state disclosure: validators reconstruct roots from explicit state payloads carried in the witness. This means the protocol still behaves as a plaintext-first protocol at validation time.

We call this the **half-Merkle problem**. A half-Merkle system simultaneously inherits:

1. the protocol complexity of commitment-bearing state;
2. the storage and privacy costs of plaintext authority;
3. the ambiguity of deciding which layer truly owns the authoritative state.

2.2 The Fee-Boundary Problem

The legacy Elone signing domain includes a fee cap. This structure implicitly encodes the proposition:

channel may pay up to some maximum fee.

But the target policy expressed in the new design note is categorically different:

channel pays no fee at all.

These are not equivalent formulations. A bounded channel-paid fee and a zero channel-paid fee induce different threat models. The former treats fee leakage as tolerable within limits; the latter treats fee leakage as a protocol violation.

2.3 The Settlement-Authorization Problem

Settlement is the point at which abstract state is turned back into spendable value. If settlement outputs are not strongly authorized, sponsor-funded fees do not solve the core problem; they merely relocate ambiguity. In particular, unilateral settlement cannot rely on ad hoc builder freedom. It requires a deterministic or explicitly committed output boundary.

2.4 Current-vs-Proposed Interpretation

Table 1: Current Snapshot vs. Proposed Architecture

Dimension	Current Snapshot	Proposed Direction
State commitment	Dual roots exist, but plaintext witness still dominates validation	Proof-first validation over committed state subtrees
Fee model	`max_fee` remains in the channel signing domain	Channel never pays fees; sponsor partition pays all fees
Settlement authorization	Output boundary is not yet the central protocol object	Timed settle is deterministic; coop settle signs a canonical output root
Local persistence	Plaintext state still behaves as local authority	Roots, proofs, signatures, and encrypted leaf bundles become authoritative
Protocol status	Partially implemented architecture	Research-grade refinement and migration target

2.5 Three Competing Truths in the Legacy Direction

The legacy direction implicitly contains three rival truth objects:

1. the on-chain roots;
2. the plaintext witness presented to validation;
3. the plaintext state snapshot retained by local software.

This is architecturally unstable. A protocol with multiple operational truths makes disagreement hard to localize: a failure may come from commitment construction, witness interpretation, or local storage drift. The Elone Prism direction intentionally collapses these truths into a single commitment-first authority:

> *the signed state header, together with admissible proofs and recovery material, is authoritative; plaintext is only cached interpretation.*

3 Research Contributions and Design Principles

3.1 Main Contributions

This paper makes the following contributions:

1. **Dual-Subtree State Commitment Model**: We formalize Elone state as separate balance and PTLC Merkle subtrees, with an optional aggregate root used as a cache key or protocol handle rather than an independent source of truth.
2. **Sponsor-Paid Fee Architecture**: We define a fee model in which channel value is never used to pay fees, and all fee expenditure is isolated in sponsor-owned wallet inputs and change outputs.
3. **Partition-Conservation Rule**: We replace fee-cap reasoning with exact value conservation over the channel partition and non-negative fee extraction from the sponsor partition.
4. **Settlement Authorization Split**: We distinguish deterministic unilateral settlement from explicitly authorized cooperative settlement and explain why these two paths should not be represented by the same authorization object.

5. **Proof-First Local Authority**: We redesign local persistence around roots, proofs, signatures, and encrypted leaves, thereby turning plaintext state from an authority object into a short-lived cache.

6. **Transition-Footprint Semantics**: We model channel transitions through explicit read/write footprints and explain why proof-first witnesses should be evaluated by touched-state locality rather than by full-state reconstruction.

7. **Recoverability as a Protocol Property**: We characterize the state package as the appropriate recovery object for unilateral enforceability, intermediate between a bare root and a full plaintext dump.

8. **Phased Migration Discipline**: We provide a migration order that prioritizes authorization repair and fee-boundary repair before witness minimization and storage refactoring.

3.2 Design Principles

The contributions above are organized by four design principles:

- **Principle 1: Merkleize only large, mutable, proof-relevant collections.** Small, hot metadata should remain outside the tree.

- **Principle 2: Value conservation is stronger than fee caps.** If channel-owned value is exactly conserved, a fee cap becomes unnecessary.

- **Principle 3: Unilateral settlement must be deterministic.** The builder should not be a hidden authorization oracle.

- **Principle 4: Local authority should be commitment-first.** Plaintext is an optimization, not the ground truth.

- **Principle 5: Merkleization should improve locality, not merely hide bytes.** The decisive gain is not compression alone, but restricting validation and debugging to the touched portion of state.

4 System Model and Threat Assumptions

4.1 Actors

We assume four actor classes:

- **Channel participants**, who jointly authorize off-chain state transitions.

- **Sponsors**, who provide ordinary wallet inputs to fund publication costs. A sponsor may be the same human or wallet operator as a channel participant, but it remains a distinct value role.

- **Consensus validators**, who decide admissibility of commitments, proofs, and value conservation.

- **Recovery runtimes**, which preserve signatures, proofs, and encrypted leaf material sufficient for continued operation or unilateral settlement.

4.2 Adversarial Goals

The adversary may attempt to:

1. make channel-owned value silently absorb publication fees;
2. exploit settlement-builder freedom to redirect unilateral outputs;
3. exploit stale plaintext authority after commitment updates;
4. manufacture proof-shape or witness-shape denial-of-service pressure.

4.3 Protocol Invariants

Invariant 4.1 (Commitment Authority). *Accepted protocol state must be derivable from the state header commitment and admissible proofs. Plaintext alone is never sufficient protocol authority.*

Invariant 4.2 (Channel Value Integrity). *No admissible Elone transaction may reduce the net value of the channel partition.*

Invariant 4.3 (Settlement Boundary Integrity). *For unilateral settlement, a valid state package and a fixed descriptor must determine a unique channel-owned output set, up to canonical serialization.*

5 Formal Transition Semantics

5.1 Channel Configuration

We model a live Elone Prism channel at logical state number t as a configuration

$$\Gamma_t = (F, H_t, \Delta, \mathcal{B}_t, \mathcal{P}_t),$$

where:

- F is the static fund anchor and channel identity context;
- H_t is the canonical state header commitment at version t;
- Δ is the settlement descriptor family fixed or versioned by the channel;
- $\mathcal{B}_t$ is the canonical set of balance leaves committed by `balances_root`;
- $\mathcal{P}_t$ is the canonical set of PTLC leaves committed by `ptlcs_root`.

Consensus never needs the full sets $(\mathcal{B}_t, \mathcal{P}_t)$ as explicit witness material. Instead, it needs only the protocol-relevant slice of those sets for the transition under evaluation. This is the key semantic shift from plaintext-first validation to proof-first validation.

5.2 Transition Footprint

Definition 5.1 (Transition Footprint). *Let $\tau : \Gamma_t \to \Gamma_{t+1}$ be an UPDATE, SETTLE, or SPLICE transition. Its **transition footprint** is the tuple*

$$\Phi(\tau) = (R_B, R_P, W_B, W_P, M, A),$$

where R_B and R_P are the balance and PTLC leaves read by the transition, W_B and W_P are the corresponding written or consumed leaves, M is the tree-external metadata consumed by validation, and A is the operation-specific authorization material.

This definition clarifies a frequently blurred issue. A channel transition does not semantically depend on "the whole state"; it depends on a bounded footprint plus the committed context in which that footprint is interpreted. Merkleization is therefore not merely a compression trick. It is a way of making the transition footprint explicit.

Definition 5.2 (Admissible Proof-First Transition)**.** *A transition $\tau : \Gamma_t \to \Gamma_{t+1}$ is* ***admissible*** *iff:*

1. *every leaf in $R_B \cup R_P$ is justified by membership, emptiness, or non-membership proofs against H_t;*
2. *the payloads in $W_B \cup W_P$ are canonically encoded and deterministically induce the next subtree roots in H_{t+1};*
3. *the authorization object A binds the operation to the relevant header commitment and settlement boundary;*
4. *the induced transaction satisfies partition conservation.*

5.3 Locality of Verification

Let $N_B = |\mathcal{B}_t|$ and $N_P = |\mathcal{P}_t|$ be the cardinalities of the balance and PTLC sets, and let $k_B = |R_B| + |W_B|$ and $k_P = |R_P| + |W_P|$ be the effective touched-leaf counts of a transition.

Proposition 5.3 (Local Verification Under a Simple Proof Model)**.** *Assume fixed-arity canonical Merkle trees, bounded tree-external metadata, and no requirement to reconstruct untouched leaves. Then verification work for an admissible proof-first transition is*

$$\mathcal{O}(k_B \log N_B + k_P \log N_P + 1),$$

rather than $\mathcal{O}(N_B + N_P)$ full-state reconstruction.

Proof. The verifier checks proofs only for the leaves named in the transition footprint. Each balance or PTLC proof has logarithmic length in the size of its corresponding canonical tree. Untouched leaves do not appear in the witness and are not reserialized. Tree-external metadata is bounded in size by protocol design. Therefore total verification work is proportional to the number of touched leaves times logarithmic proof length, plus constant-sized metadata checks. □

The academic significance of this theorem is broader than asymptotics. Locality narrows the epistemic surface of failure. When a transition is invalid, the disagreement is localized to the named footprint, its proofs, or its authorization boundary. In a plaintext-first system, disagreement can hide inside a much larger reconstructed state snapshot.

6 Merkleized State Commitment Model

6.1 Dual Subtrees and Optional Aggregate Root

We do not recommend a single flat state tree. Balance entries and PTLC entries have different update locality, different verification patterns, and different high-frequency predicates. In particular, the predicate

$$\texttt{ptlcs_root} = \texttt{EMPTY_PTLCS_ROOT}$$

is operationally significant and should remain cheap to evaluate.

The proposed structure is:

$$\texttt{state_root} = \mathrm{H}(\texttt{version} \parallel \texttt{balances_root} \parallel \texttt{ptlcs_root}).$$

The roots `balances_root` and `ptlcs_root` remain the semantically meaningful commitments. The optional `state_root` is a derived aggregate handle. It must never become a competing source of truth.

Definition 6.1 (State Header Commitment)**.** *The* ***state header commitment*** *is the canonical hash of the protocol-visible channel state header:*

$$\begin{aligned}\mathrm{H}(&\mathit{chain_id} \parallel \mathit{channel_id} \parallel \mathit{fund_outpoint} \parallel \mathit{participants_root}\\ &\parallel \mathit{state_number} \parallel \mathit{balances_root} \parallel \mathit{ptlcs_root}\\ &\parallel \mathit{state_layout_version} \parallel \mathit{settlement_descriptor_root}).\end{aligned}$$

6.2 Tree-Internal vs. Tree-External Data

The design memorandum correctly distinguishes *large, mutable collections* from *small, hot metadata*. The former should be Merkleized; the latter should remain tree-external.

Table 2: Recommended State Placement

Category	Examples	Recommended Placement
Mutable state sets	balance leaves, PTLC leaves, settlement leaf material	Merkle subtrees
Hot metadata	`channel_id`, `state_number`, `csv_delay`, `participants_root`	Tree-external fields
Protocol versioning	`state_layout_version`, descriptor root	Tree-external, but committed via the state header
Wallet execution detail	sponsor inputs, sponsor change, coin selection trace	Never inside the channel state tree

6.3 State Carrier Reserve

The value previously described operationally as `state_utxo_value` should not be treated as business logic or fee budget. Under the proposed model it is demoted to a technical reserve:

$$\texttt{state_carrier_reserve} = \max(\texttt{MIN_STATE_UTXO_VALUE}, \texttt{layout_specific_minimum}).$$

Its role is purely to satisfy base-layer state-carrier requirements. It must:

1. remain outside the signing semantics of off-chain balance evolution;
2. never be interpreted as channel fee budget;
3. be fully conserved or refunded according to transaction type.

6.4 Canonical Leaf Discipline

Merkleization is not merely a storage compression technique; it is a semantics discipline. Let each balance leaf and PTLC leaf carry a canonical ordering key determined by protocol rules rather than by local insertion order. Then:

$$\begin{aligned}\texttt{balances_root} &= \mathrm{H}(\texttt{sorted balance leaves}),\\ \texttt{ptlcs_root} &= \mathrm{H}(\texttt{sorted PTLC leaves}).\end{aligned}$$

This canonicalization requirement has an important theoretical consequence: the protocol ceases to interpret Merkle roots as opaque hashes and instead interprets them as *canonical summaries of a uniquely ordered state set*. Without canonical leaf order, two semantically identical local states could still diverge at the commitment layer.

6.5 Proof Objects as First-Class Protocol Objects

The proof-first model requires more than "some Merkle proofs." We distinguish three first-class protocol objects:

1. **StateAccessManifest**: the declaration of which subtrees and leaves are relevant to the transition;
2. **MerkleProofBundle**: the membership, emptiness, or non-membership proofs that justify those declarations;
3. **LeafPayloadBundle**: the canonical payload material whose hashes realize the claimed leaves.

The academic insight here is that a proof-first channel protocol should treat *proof shape* as part of protocol meaning, not merely as SDK serialization. Once state access becomes partial and structured, the witness itself becomes a constrained proof object rather than a bag of implementation detail.

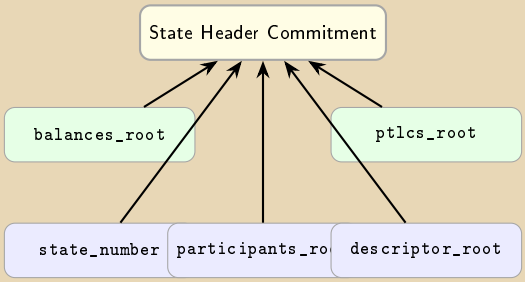

Figure 1: Proof-first Elone state: large mutable sets are Merkleized, while hot metadata remains tree-external but commitment-bound.

7 Sponsor-Paid Fees and Partition Conservation

7.1 Initialization vs. Post-Funding Transitions

FUND is not semantically identical to UPDATE, SETTLE, or SPLICE. FUND creates the initial channel partition from ordinary wallet-owned value. By contrast, UPDATE, SETTLE, and SPLICE transform an already existing channel partition. This distinction matters because exact channel-partition conservation is a property of *post-funding channel transitions*, not of the initial funding act itself.

We therefore distinguish:

- **initialization**: FUND creates channel-owned outputs from setup-wallet value and pays fees in the ordinary wallet manner;
- **post-funding transitions**: UPDATE, SETTLE, and SPLICE preserve the already created channel partition while allowing sponsor-funded publication.

7.2 Partitioned Value Semantics

Every post-funding Elone transition is divided into two value partitions:

- the **channel partition**, consisting of channel-owned inputs and channel-owned outputs;
- the **sponsor partition**, consisting of ordinary wallet inputs and ordinary sponsor change outputs.

This division is semantic rather than script-template-based. In particular, cooperative settlement may produce ordinary address outputs that nonetheless belong to the channel partition. Therefore, partition membership cannot be determined by superficial output shape. It must be determined by commitment membership or deterministic derivation.

7.3 Partition Assignment Semantics

Definition 7.1 (Partition Map). *For each post-funding Elone transition T, the protocol defines a deterministic map*

$$\pi_T : E(T) \to \{C, S, R\},$$

where $E(T)$ is the set of transaction-relevant elements, C denotes channel-owned value, S denotes sponsor-owned value, and R denotes reference-only elements that affect validation but do not enter value conservation.

The crucial role of π_T is to prevent partition assignment from degenerating into output-shape heuristics. The map is part of protocol semantics, not wallet convention.

Table 3: Deterministic Partition Assignment for Post-Funding Transitions

Tx Type	**Channel-Owned (C)**	**Sponsor-Owned (S)**	**Reference-Only (R)**
UPDATE	consumed `ChannelState`; created next `ChannelState`	sponsor wallet inputs and sponsor change outputs	`ChannelFundRef` context input
Timed SETTLE	consumed `ChannelFund` and `ChannelState`; all outputs from descriptor derivation	optional sponsor wallet inputs and sponsor change outputs	none
Coop SETTLE	consumed `ChannelFund` and `ChannelState`; outputs committed by `settle_outputs_root`	optional sponsor wallet inputs and sponsor change outputs not covered by that root	none
SPLICE	all consumed and created channel-typed topology outputs designated by splice commitments	sponsor wallet inputs and sponsor change outputs	topology references with no spend value

FUND is exceptional. It creates the initial channel partition by moving setup-wallet value into `ChannelFund` and `ChannelState(0)` outputs. Fees are still wallet-funded, but exact conservation is not yet defined over a pre-existing channel partition because none exists before funding.

Definition 7.2 (Partition Conservation). *Let T be a post-funding Elone transition. Let $V_C^{in}(T)$ and $V_C^{out}(T)$ denote the total input and output value of the elements for which $\pi_T(e) = C$. Let $V_S^{in}(T)$ and $V_S^{out}(T)$ denote the corresponding totals for the elements for which $\pi_T(e) = S$. The transition satisfies* ***partition conservation*** *iff:*

$$V_C^{in}(T) = V_C^{out}(T)$$

and

$$V_S^{in}(T) \geq V_S^{out}(T).$$

The fee is then defined as:

$$\mathrm{Fee}(T) = V_S^{in}(T) - V_S^{out}(T).$$

Proposition 7.3 (Channel Fee Isolation). *If partition conservation holds, then channel-owned value never pays transaction fees.*

Proof. By definition, $V_C^{in}(T) = V_C^{out}(T)$. Therefore no value deficit is introduced within the channel partition of T. Since total fee is defined entirely by the sponsor deficit $V_S^{in}(T) - V_S^{out}(T)$, all fees must be paid by the sponsor partition. Hence channel-paid fee is identically zero. □

Corollary 7.4. *Under partition conservation, a channel fee cap is not a primary safety invariant.*

7.4 Why Fee Caps Are Insufficient

A fee cap bounds channel loss; it does not prohibit channel loss. The sponsor-paid model aims to forbid channel fee expenditure rather than merely constrain it. This is a categorical difference in protocol semantics. Consequently:

1. deleting `max_fee` without partition conservation is unsafe;
2. keeping `max_fee` after partition conservation is conceptually redundant.

7.5 Why Sponsorship Remains Central Even If the Title Changes

This paper does not foreground sponsor-paid fees in its title because the deeper conceptual object is the merkleized channel itself. Nevertheless, sponsorship remains central to the protocol logic. Once channel-owned value is defined to be exactly conserved, some other partition must bear publication cost. The sponsor abstraction is therefore not cosmetic wallet terminology; it is the value-theoretic complement of channel conservation.

8 Transaction Semantics Under the New Model

8.1 FUND

FUND remains the least controversial path. It already uses wallet-owned inputs and can therefore continue to fund its own fees in the usual base-layer manner. The important change is architectural: FUND should fix the long-lived settlement descriptor and state layout anchors early, rather than leaving settlement boundary semantics implicit.

8.2 Legacy vs. Elone Prism Semantics

Table 4: Legacy Direction vs. Elone Prism

Aspect	**Legacy Direction**	**Elone Prism Direction**
State authority	Plaintext snapshots dominate operational truth	State headers and proofs dominate operational truth
Witness model	Plaintext-assisted	Access-scoped and proof-first
Fee semantics	Channel loss is capped	Channel loss is prohibited by conservation
Settlement semantics	Builder freedom remains substantial	Descriptor or output-root binds the boundary
Recovery object	Latest local state dump	State package with proofs and encrypted leaves

8.3 UPDATE

The proposed UPDATE consumes the previous state carrier and references the fund anchor:

- channel inputs: `ChannelStateSpend` and `ChannelFundRef`;
- sponsor inputs: ordinary wallet UTXOs, if fees must be paid;
- outputs: the new `ChannelState` plus sponsor change.

The state carrier reserve is preserved:

$$\texttt{old_state_value} = \texttt{new_state_value}.$$

The fee is paid exclusively by sponsor inputs.

```rust
pub struct UpdateWitnessVNext {
    pub state_header_commitment: [u8; 32],
    pub access_manifest: StateAccessManifest,
    pub proof_bundle: MerkleProofBundle,
    pub touched_leaf_payloads: Vec<Vec<u8>>,
    pub agg_signature: Signature,
}
```

Listing 1: Proof-First UPDATE Witness Sketch

8.4 SETTLE

SETTLE is the most security-sensitive path because it converts committed state into spendable outputs. We separate two modes:

Timed / unilateral settle. The unilateral path should not permit arbitrary builder freedom. Its outputs must be uniquely derivable from:

1. the proved channel state;
2. the participants commitment;
3. the settlement descriptor.

Proposition 8.1 (Deterministic Timed Settle)**.** *Given a valid state proof, a fixed settlement descriptor, and a canonical serialization rule, the unilateral settlement output set is unique.*

Proof sketch. The descriptor specifies the authorized settlement transformation. The proved leaves determine balances and any required PTLC emptiness predicates. Canonical serialization removes builder discretion. Therefore the resulting channel-owned output multiset is uniquely determined. □

Cooperative settle. The cooperative path may intentionally expose more flexibility for address rotation, output coalescing, or dust management. That flexibility must be bounded by an explicit channel-owned output commitment:

$$\texttt{settle_outputs_root}.$$

Sponsor change outputs must remain outside this commitment.

This split matches an important asymmetry already visible in the current Elone validator. Timed settle enforces commitment consistency against the on-chain state carrier, whereas cooperative settle can authorize a later mutually signed off-chain state without first publishing an intermediate UPDATE. Elone Prism does not erase that asymmetry; it makes it explicit and gives each path a distinct authorization object.

8.5 Settlement Descriptor as a Protocol Object

The settlement descriptor is the bridge between abstract state commitments and concrete value redistribution. A useful descriptor should specify:

1. participant destination policy;
2. unilateral derivation rules;
3. cooperative canonicalization rules;
4. versioning hooks for future settlement evolution.

This yields a clean academic separation:

- **Timed settle**: the descriptor acts as an algorithmic specification.
- **Coop settle**: the descriptor constrains a separately signed `settle_outputs_root`.

The descriptor is thus neither a mere wallet hint nor a generic scripting language. It is a narrow authorization object whose purpose is to eliminate hidden builder discretion.

8.6 A Minimal Descriptor Calculus

To make the descriptor auditable, we define a minimal descriptor grammar:

$$\Delta = (\texttt{version}, \texttt{participants}, \rho_{\text{pay}}, \rho_{\text{ptlc}}, \rho_{\text{asset}}, \rho_{\text{coop}}),$$

where:

- `participants` fixes the participant set and destination identities available to the channel;
- ρ_{pay} maps proved balance leaves to canonical payout instructions;
- ρ_{ptlc} specifies the PTLC emptiness or resolution predicates required before timed settlement is admissible;
- ρ_{asset} specifies asset-aware routing and output grouping policy;
- ρ_{coop} specifies the canonicalization rule for cooperative output commitments.

The point of this calculus is not to create a general-purpose scripting language. It is to specify precisely which degrees of freedom are allowed to influence settlement and which are not.

8.7 Descriptor Commitment and Upgrade Boundaries

The descriptor root belongs in the state header commitment because it changes the meaning of settlement, not merely its serialization. Two consequences follow:

1. if the descriptor root changes, the channel has changed one of its authorization objects, and the change must therefore be reflected in a new authorized state header;
2. a state package remains recoverable across time only relative to the descriptor version committed by that package.

This is why descriptor upgrades cannot be treated as wallet-local metadata edits. They are semantic upgrades to the settlement calculus itself.

```
fn derive_timed_settle_outputs(
    package: &StatePackage,
    descriptor: &Descriptor
) -> Result<Vec<Output>, Error> {
    require(package.descriptor_root == descriptor.commitment());
    require(descriptor.ptlc_rule.admits(&package.ptlc_proofs)?);

    let mut outputs = vec![];
    for leaf in package.balance_leaves.iter().sorted_by_key(canonical_key) {
        outputs.push(descriptor.payout_rule.to_output(leaf)?);
    }

    descriptor.asset_policy.normalize(&mut outputs)?;
    descriptor.coop_policy.assert_not_required()?;
    canonical_serialize(outputs)
}
```

Listing 2: Timed-Settle Output Derivation Sketch

8.8 Timed-Settle Derivation

The minimal deterministic derivation rule can be expressed as follows.

The important fact about this pseudocode is not its surface syntax. It is that no hidden builder discretion remains once the state package and descriptor are fixed. That property is exactly what unilateral settlement needs.

8.9 SPLICE

SPLICE preserves exact channel-owned value across channel topology transitions while permitting sponsor-funded fees. The semantic equation is:

$$\sum \text{consumed channel-owned values} = \sum \text{created channel-owned values.}$$

From an engineering perspective, SPLICE should not be the first path to receive maximal proof optimization. UPDATE and SETTLE should be stabilized first, because SPLICE magnifies every unresolved boundary in the system.

Table 5: Transaction-Level Effect of the New Model

Tx Type	Channel Semantics	Sponsor Semantics
FUND	Creates `ChannelFund` and `ChannelState(0)`	Ordinary wallet funding path
UPDATE	State carrier reserve migrates 1:1 into the next state	Sponsor covers fee and receives sponsor change
SETTLE	All channel-owned value must appear in authorized channel settlement outputs	Optional sponsor covers fee only
SPLICE	Channel-owned value is exactly redistributed across topology outputs	Sponsor covers fee without altering net channel value

9 Proof-First Witness and Local Persistence

9.1 From Plaintext Authority to Commitment Authority

The local authority object should no longer be a pair of plaintext vectors:

- `Vec<BalanceEntry>`

- `Vec<PtlcEntry>`

Instead, the authoritative local object becomes:

- the latest state number;
- the latest signed state header commitment;
- proof inventory;
- signature inventory;
- encrypted leaf material sufficient for recovery.

9.2 Recommended Storage Layers

We adopt the following four-layer local storage model:

1. **Channel tip metadata:** state number, roots, signatures, and descriptor version.
2. **Merkle node cache:** optional node material indexed by hash.
3. **Encrypted leaf bundle store:** serialized balance/PTLC/descriptor leaf payloads.
4. **Secret vault:** adaptor material, revealed secrets, and local recovery-only secrets.

The paper does not recommend an aggressively abstract graph database for Merkle nodes as the first implementation target. Correctness, recoverability, and predictable garbage collection are more important than maximal generality in version one.

9.3 State Package as the New Recovery Unit

To make commitment authority operational, we define a **state package** as the minimal recoverable object that permits a participant to continue operating or to settle:

1. the state header commitment and version fields;
2. the latest aggregated signatures needed for admissible next actions;
3. the proofs required for those next actions;
4. the encrypted leaf material required to reconstruct or interpret those proofs.

This object is richer than a bare root and leaner than a permanent plaintext state dump. That is precisely why it is the correct recovery object: it preserves protocol meaning without reintroducing plaintext authority as a long-lived invariant.

9.4 Privacy Interpretation

The goal is not "never persist plaintext in any circumstance." The realistic goal is stronger and more operational:

1. plaintext should not be the default authority object;
2. crash recovery should not require plaintext snapshots;
3. any plaintext cache should be local, short-lived, and encrypted when persisted.

10 Recoverability as a Protocol Property

10.1 Bare Roots, Plaintext Dumps, and State Packages

A bare root is compact, but semantically thin. A plaintext dump is semantically rich, but operationally heavy and privacy-hostile. The Elone Prism state package is intentionally placed between these extremes.

Proposition 10.1 (Bare-Root Insufficiency)**.** *If unilateral settlement depends on committed balances, PTLC emptiness predicates, or descriptor-governed payout rules, then the state header commitment alone is insufficient to construct a valid unilateral settlement transaction.*

Proof. The state header commitment authenticates the existence of the relevant subtree roots and metadata, but it does not reveal the leaf payloads or proofs required to interpret those roots. Deterministic unilateral settlement depends on the actual committed balances and on any PTLC predicates that must be discharged before settlement. Therefore a party holding only the header commitment cannot derive the complete unilateral settlement output set. □

Proposition 10.2 (State-Package Sufficiency for Unilateral Recovery)**.** *Suppose a participant stores the latest state package consisting of the state header commitment, the current settlement descriptor version, the signatures needed for admissible unilateral publication, the settle-relevant proofs, and decryptable leaf payloads for those proofs. Then the participant can reconstruct the deterministic unilateral settlement outputs for that state.*

Proof. The stored header commitment identifies the authoritative state version. The descriptor fixes the unilateral derivation rule. The settle-relevant proofs authenticate the balance and PTLC data required by that rule, and the stored signatures authorize publication. Because unilateral settlement is deterministic under the descriptor, these materials are sufficient to derive the unique channel-owned output multiset. □

These two propositions explain why the state package is the correct recovery object. It is strictly richer than a root, because a root alone cannot operationalize settlement. It is strictly leaner than a permanent plaintext dump, because only the leaves and proofs required for admissible next actions must be retained as authority material.

10.2 Authority Compression

This motivates a deeper interpretation of Merkleization. The point is not merely to replace a large vector with a shorter hash. The point is to *compress authority.* A plaintext-first design says that the whole decoded state is the thing one must preserve forever. A proof-first design says that only the committed recovery frontier must be preserved forever. Everything else becomes a cache policy, not a protocol obligation.

10.3 Failure Domains Under the New Model

Proof-first recovery also sharpens failure analysis. A participant can now distinguish at least four classes of recoverability failure:

1. **missing authorization**: signatures for the latest admissible publication are absent;
2. **missing proof material**: the relevant Merkle paths cannot be reconstructed;
3. **missing decryptability**: encrypted leaf blobs exist but cannot be opened;
4. **missing sponsor liquidity**: publication is authorized but cannot be funded.

This decomposition is an academic gain as well as an engineering one. It turns "recovery failed" from a monolithic event into a structured diagnostic space, which is exactly what a mature protocol theory should provide.

11 Signature Domains and Authorization Boundaries

11.1 State-Header-Centric Signing

The design note correctly warns against signing a long, loosely organized list of scattered fields. We therefore recommend a state-header-centric signing discipline. Operation-specific messages should sign:

1. channel identity and chain context;
2. the canonical `state_header_commitment`;
3. operation-specific authorization material.

11.2 UPDATE vNext

UPDATE signatures should bind:

- the state header commitment;
- any retained validity window;
- optionally, a proof commitment if proof canonicalization becomes part of the signed object.

They should not bind:

- sponsor coin selection;
- sponsor outputs or sponsor change;
- wallet-specific fee-bump artifacts.

11.3 SETTLE vNext

SETTLE signatures should bind:

- the state header commitment;
- the settle mode;
- for timed settle, a descriptor commitment or equivalent deterministic output derivation rule;
- for cooperative settle, the canonical `settle_outputs_root`.

This yields a clean boundary:

1. the channel authorizes its own output set or output derivation rule;
2. the sponsor authorizes its own funding and change path.

11.4 SPLICE vNext

SPLICE signatures should bind the old channel identity, new topology commitments, and any descriptor or layout delta that affects downstream semantics. Sponsor funding details remain outside the channel signing boundary.

11.5 Proof Commitments and Signature Minimality

An unresolved but important question is whether proof objects themselves should enter the signed domain. A defensible rule is:

1. always sign the state header commitment;
2. sign proof commitments only when proof shape affects semantic interpretation;
3. never sign sponsor coin-selection detail as part of channel authorization.

The academic point is that *signature minimality* matters: once signatures absorb wallet mechanics, channel authorization and wallet execution become inseparable again.

12 Evaluation Agenda and Measurable Claims

12.1 What This Draft Does and Does Not Claim

This draft now specifies the protocol objects sharply enough to support benchmarking, but it does not yet report benchmark results. That limitation matters. Without empirical measurements, Elone Prism remains a protocol thesis with formalized objects rather than a completed systems paper.

12.2 Hypotheses for Empirical Evaluation

The design yields four primary hypotheses:

1. **Witness-size hypothesis**: proof-first witnesses become smaller than plaintext witnesses once balance/PTLC sets exceed a protocol-dependent threshold.
2. **Verification-locality hypothesis**: verifier work tracks touched leaves and proof overlap rather than full state cardinality.
3. **Recovery-object hypothesis**: a state package is materially smaller than an always-persisted plaintext snapshot while still sufficient for unilateral recovery.
4. **DoS-boundary hypothesis**: adversarial proof shapes can be bounded by manifest constraints and canonical proof policies.

12.3 Minimum Benchmark Matrix

Table 6: Minimum Benchmark Program Required for a Systems Version of Elone Prism

Dimension	**Parameter Sweep**	**Metrics**
State size	32 / 128 / 512 balance leaves; 0 / 16 / 64 PTLC leaves	witness bytes, proof bytes, output bytes
Verification cost	UPDATE / timed SETTLE / coop SETTLE / SPLICE	CPU time, hash count, cache-hit ratio
Recovery cost	bare root vs. state package vs. encrypted plaintext snapshot	recovery latency, persisted bytes, unilateral-settle readiness
Adversarial stress	worst-case touched-leaf pattern, overlapping proofs, sponsor-fee spikes	verifier slowdown, mempool rejection rate, failure classification

12.4 Why This Matters

The benchmark agenda is not decorative. It is the empirical bridge between the protocol thesis of this paper and a systems paper claim. Until those measurements exist, locality, recoverability, and DoS resilience should be read as structured hypotheses with supporting design arguments rather than as experimentally closed results.

13 Related Work and Positioning

13.1 Payment Channels, Update-by-Replacement, and Factories

Lightning-style payment channels frame channel security around revocation or punishment logic and off-chain state replacement [1]. Eltoo reframes that lineage by using update-by-replacement and ANYPREVOUT-style semantics to make the latest state authoritative [2, 3]. Channel factories extend the same family toward shared funding and nested topology management [4]. Elone Prism is adjacent to this literature, but its main claim is different: the novel issue is not merely how to replace states, but how to relocate authority from plaintext snapshots to proof-carrying state packages while separating publication fees from conserved channel value.

PTLC-oriented constructions and anonymous multi-hop locks extend channel expressiveness beyond HTLC-style lock semantics [5]. Elone Prism does not compete with that work; rather, it asks how PTLC-bearing state should be committed, proved, and recovered once PTLC sets themselves become Merkleized protocol objects.

13.2 Merkleized State and Witness-Carrying Validation

Merkleized and proof-carrying validation ideas are well established in blockchain research. Utreexo shows how authenticated accumulators can move large validation state into compact proofs [6]. Plasma-style exit systems and related off-chain constructions show how committed state and explicit exit proofs can be combined to support conditional withdrawal from shared state [7]. Elone Prism differs in two ways. First, its witnesses are not global-state proofs but bilateral channel-transition footprints. Second, its goal is not merely stateless verification; it is to redefine the authority object used by the channel runtime itself.

13.3 Fee Sponsorship and Publication Economics

Transaction sponsorship and fee abstraction are familiar in account-abstraction systems, for example through paymaster-style relaying models such as ERC-4337 [8]. Elone Prism borrows the intuition that the actor paying publication cost need not be identical to the actor whose state transition is being published. Its contribution is to internalize that separation as a channel-level invariant: sponsor funding is not an implementation convenience but a partition assignment rule checked by protocol semantics.

13.4 What Elone Prism Adds Beyond Generic Merkleization

Many systems can place state under a Merkle root. That fact alone does not identify the contribution of Elone Prism. The distinctive move here is the simultaneous separation of three boundaries:

1. **state boundary**: roots and proofs replace plaintext snapshots as authority;
2. **value boundary**: channel-owned value is conserved independently of sponsor-funded fees;
3. **authorization boundary**: unilateral and cooperative settlement are given different commitment objects.

This combination is what turns merkleized state from a storage convenience into a protocol refinement.

13.5 Comparative Axes

Table 7: Conceptual Positioning of Elone Prism

Axis	Penalty Channels	Latest-State Channels	Global Merkleized Systems	Elone Prism
Authority object	Revocation history plus latest state	Latest signed state	Global state root plus operator proofs	State header commitment plus transition-specific proofs
Fee boundary	Commonly coupled to channel spend path	Often still channel-coupled	Operator-funded batch publication	Sponsor partition separated from conserved channel value
Settlement boundary	Script or builder driven	Simplified, but often output-coupled	Batch exit rules	Deterministic unilateral settle or explicit coop output root
Witness granularity	Full state or script path	Latest state focused	Batch / shared-state proof	Access-scoped channel footprint

The table makes an important academic point. Elone Prism is not best described as "Lightning with Merkle trees" or as "a channel-flavored rollup." It is a protocol in which bilateral channel semantics, native transaction typing, and proof-local witnesses are co-designed. That co-design is what permits sponsor separation and deterministic settlement to remain crisp rather than accidental.

13.6 A Distinctive Research Claim

The most distinctive research claim of this paper is the following:

> *Merkleization is only protocol-progressive when it reduces the authority surface and isolates value and authorization boundaries at the same time.*

Without that joint isolation, Merkle roots are too easy to deploy as ornamental compression: the hashes exist, but plaintext still dominates execution, fees still blur across partitions, and settlement still hides builder discretion. Elone Prism is an argument against that ornamental form of Merkleization.

14 Migration Roadmap and Open Engineering Questions

14.1 Recommended Migration Order

The proposed migration order is:

1. **Phase A: Repair settlement authorization.** Introduce explicit or deterministic settlement output binding.

2. **Phase B: Adopt sponsor-only fees.** Remove `max_fee`, admit sponsor wallet inputs, and enforce partition conservation.

3. **Phase C: Move to proof-first witnesses.** Replace plaintext-assisted validation with proof-assisted validation.

4. **Phase D: Remove plaintext authority from local persistence.** Promote roots, proofs, and encrypted leaves to authoritative storage.

This order is important. Witness minimization should not precede authorization repair. Storage refactoring should not precede consensus-level semantic convergence.

14.2 Open Question 1: Liveness Under Sponsor Dependence

Sponsor-only fees improve safety but introduce a liveness obligation. Each participant must maintain an emergency sponsor reserve large enough to publish force-update and timed-settle transactions without relying on the counterparty or an external service.

A natural policy formula is:

$$\begin{aligned}\texttt{emergency_sponsor_reserve} = {} & \texttt{fee_budget(force_update)} \\ & + \texttt{fee_budget(timed_settle)} \\ & + 2 \cdot \texttt{fee_budget(rbss_bump)} \\ & + \texttt{consolidation_margin}.\end{aligned}$$

This reserve formula becomes meaningful only under an explicit fee-spike model. Let λ denote the prevailing fee rate and let $\lambda_{\max}$ denote the adversarial fee rate the protocol chooses to survive. Then a minimal liveness condition for unilateral publishability is:

$$\texttt{emergency_sponsor_reserve} \geq \sum_{a \in \mathcal{A}_{\text{unilateral}}} \texttt{mass}(a) \cdot \lambda_{\max} + \texttt{rebroadcast_margin},$$

where $\mathcal{A}_{\text{unilateral}}$ contains every action a participant may need to publish in the worst credible recovery path.

This observation is important for academic positioning. Partition conservation solves a *safety* problem by forbidding channel-paid fees. It does not by itself solve the *liveness* problem of getting an authorized transaction mined during fee volatility. Any mature version of Elone Prism therefore needs one of three publication strategies:

1. **self-funded reserve**: each participant maintains a locally controlled sponsor reserve sized for $\lambda_{\max}$;

2. **bonded fee lane**: a pre-committed sponsor pool is reserved for emergency publication only;

3. **third-party sponsorship**: an external sponsor or service is authorized to pay fees without changing the channel partition.

If none of these strategies is in place, the protocol may be safe in the narrow sense of value conservation while still failing operationally under fee spikes. That distinction should be read as a central research constraint, not an implementation footnote.

14.3 Open Question 2: Proof Size and DoS Envelope

Proof-first witnesses should not be evaluated only for correctness. They must also be benchmarked for:

1. witness size under realistic balance/PTLC distributions;

2. verifier cost under adversarial touched-leaf patterns;

3. mempool and block-level DoS implications.

14.4 Open Question 3: Descriptor Upgrade Flexibility

Settlement descriptors must be rigid enough to authorize unilateral settlement, but not so rigid that all future settlement evolution requires a protocol fork. The right object is therefore neither a free-form builder language nor a completely frozen output template. It is an upgradeable, versioned descriptor with explicit canonical derivation rules.

14.5 Open Question 4: Factory-Level Composition

This paper treats partition conservation primarily at the single-channel semantic level. In nested splice and factory topologies, however, reserve fragmentation, descriptor inheritance, and sponsor allocation become coupled. A complete Elone Prism theory should therefore explain:

1. how reserve values decompose across subchannels;
2. how descriptor roots are inherited or replaced during topology changes;
3. how sponsor liquidity is budgeted across multiple simultaneously publishable states.

15 Conclusion

This paper has argued that the Merkleization of Elone state cannot be separated from fee responsibility and settlement authorization. A protocol with roots but plaintext authority is incomplete. A protocol without `max_fee` but without partition conservation is unsafe. A sponsor-funded settlement flow without explicit output boundaries is under-authorized.

The proposed architecture resolves these problems by combining:

1. dual-subtree state commitments;
2. sponsor-paid fees;
3. exact partition conservation;
4. deterministic unilateral settlement;
5. proof-first witnesses and local persistence.

The resulting design is not merely "more Merkleized." It is a cleaner separation of responsibilities across the protocol stack. Consensus validates proofs and value conservation. Channel signatures authorize state and channel-owned output boundaries. Sponsors fund publication costs. Local storage preserves recoverability without treating plaintext as the permanent source of truth.

The March 2026 design snapshot should therefore be understood as a decisive architectural refinement for Elone. Under the name **Elone Prism**, it is best read as the merkleized, proof-first specialization of the broader Elone program. Its academic contribution is not just a new state layout. It is the claim that channel protocols become cleaner when they are analyzed through three separations at once: authority from cache, channel value from publication fees, and deterministic settlement from cooperative flexibility.

That thesis is already coherent enough to function as the main protocol line, but it becomes genuinely mature only if four engineering closures are completed: sponsor-liveness reserves, proof-size and DoS benchmarking, upgradeable settlement descriptor design, and factory-level composition semantics. Those are not peripheral implementation chores. They are the final steps required to turn Elone Prism from a sharp protocol thesis into a fully operational merkleized channel architecture.

References

[1] Joseph Poon and Thaddeus Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments.* White paper, 2016. https://nakamotoinstitute.org/library/lightning-network/

[2] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. *eltoo: A Simple Layer2 Protocol for Bitcoin.* Technical report, 2018. https://blockstream.com/eltoo.pdf

[3] Christian Decker and Anthony Towns. *BIP 118: SIGHASH_ANYPREVOUT for Taproot Scripts.* Draft specification, 2017. https://bips.dev/118/

[4] Conrad Burchert, Christian Decker, and Roger Wattenhofer. *Scalable Funding of Bitcoin Micropayment Channel Networks.* Royal Society Open Science, 5(8):180089, 2018. https://royalsocietypublishing.org/doi/10.1098/rsos.180089

[5] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, and Matteo Maffei. *Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability.* In *NDSS*, 2019. https://www.ndss-symposium.org/ndss-paper/anonymous-multi-hop-locks-for-blockchain-scalability-and-interoperability/

[6] Thaddeus Dryja. *Utreexo: A Dynamic Hash-Based Accumulator Optimized for the UTXO Set.* IACR ePrint 2019/611, 2019. https://eprint.iacr.org/2019/611

[7] Joseph Poon and Vitalik Buterin. *Plasma: Scalable Autonomous Smart Contracts.* White paper, 2017. https://plasma.io/plasma.pdf

[8] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. *ERC-4337: Account Abstraction Using Alt Mempool.* Draft standard, 2021. https://eips.ethereum.org/EIPS/eip-4337